

Construindo o “R” do RAG: Um Sistema de Recuperação de Artigos Científicos sobre Engenharia de Software Cross-Platform

Diego Rodrigues de Souza Arimura¹

¹Faculdade de Computação (FACOM) – Universidade Federal de Mato Grosso do Sul (UFMS)
Caixa Postal 549 – 79.070-900 – Campo Grande – MS – Brazil

Abstract. *Retrieval-Augmented Generation (RAG) has emerged as a cornerstone for building reliable question-answering systems over specific knowledge bases. In this context, the retrieval component (the “R”) is responsible for selecting the most relevant documents to feed the generative model. This work designs, implements, and evaluates a document retriever on a scientific paper collection from ArXiv containing 2,687 documents related to cross-platform mobile software engineering (Flutter, React Native, Android, and iOS). We compare three retrieval strategies: a lexical sparse baseline (BM25), a dense semantic retriever (KNN using all-MiniLM-L6-v2 embeddings), and a hybrid retriever utilizing Reciprocal Rank Fusion (RRF). Evaluating these models over 15 targeted queries shows that the hybrid approach balances the strengths of both representations, achieving the best overall Recall@10 (0.5685) and Top Precision@10 (0.5333).*

Resumo. *A Geração Aumentada de Recuperação (RAG) consolidou-se como uma abordagem fundamental para construir sistemas confiáveis de perguntas e respostas sobre bases de conhecimento específicas. Nesse contexto, o componente de recuperação (o “R”) é responsável por selecionar os documentos mais relevantes para alimentar o modelo gerador. Este trabalho projeta, implementa e avalia um recuperador de documentos sobre uma coleção de 2.687 artigos científicos extraídos do ArXiv relacionados à engenharia de software móvel cross-platform (Flutter, React Native, Android e iOS). Comparamos três estratégias: um baseline esparso léxico (BM25), um recuperador denso semântico (KNN usando embeddings all-MiniLM-L6-v2) e um recuperador híbrido utilizando Reciprocal Rank Fusion (RRF). A avaliação dos modelos sobre 15 queries direcionadas demonstra que a abordagem híbrida balanceia as forças de ambas as representações, obtendo a melhor cobertura Recall@10 (0.5685) e Precisão@10 de topo (0.5333).*

1. Introdução

A produção literária na área de tecnologia da informação tem crescido a taxas exponenciais, criando um gargalo considerável para pesquisadores e profissionais no momento de identificar trabalhos relevantes. No domínio do desenvolvimento móvel, o advento de tecnologias de desenvolvimento híbrido e multiplataforma (*cross-platform*), como Flutter (baseado na linguagem Dart) e React Native (baseado em JavaScript/TypeScript), gerou uma grande quantidade de literatura acadêmica enfocando tópicos como testes automatizados, consumo energético, qualidade de código e desempenho em relação às plataformas nativas (Android/Java/Kotlin e iOS/Swift).

Em paralelo, a ascensão dos Grandes Modelos de Linguagem (LLMs) proporcionou novas capacidades de interação em linguagem natural com bases de dados. Contudo, esses modelos sofrem com alucinações e a limitação de conhecimento temporal ou privado. Para contornar esse desafio, a arquitetura de Geração Aumentada por Recuperação (RAG - *Retrieval-Augmented Generation*) tornou-se o padrão da indústria. Em um sistema RAG, antes que a fase generativa seja iniciada, um componente de busca — o “R” — atua sobre um corpus pré-indexado para extrair um conjunto conciso de documentos semanticamente relevantes à consulta. Caso este componente falhe e entregue documentos ruidosos ou irrelevantes, a qualidade da resposta gerada decairá severamente.

Este trabalho aborda o desenvolvimento, a avaliação e a análise conceitual de um sistema de recuperação de artigos científicos do ArXiv focado em Engenharia de Software Móvel Cross-Platform. Investigamos os limites e as forças de abordagens léxicas tradicionais baseadas em termos exatos (BM25 Okapi), técnicas de recuperação semântica densa baseadas em embeddings de aprendizado profundo (KNN com vetores densos) e estratégias híbridas que fundem os dois paradigmas utilizando o algoritmo Reciprocal Rank Fusion (RRF).

O trabalho está estruturado da seguinte forma: a Seção 2 revisa trabalhos relacionados na área de recuperação de informação léxica e semântica; a Seção 3 detalha a metodologia de coleta, pré-processamento e modelagem do retriever; a Seção 4 descreve a avaliação experimental e o mecanismo de anotação (*pooling*); a Seção 5 expõe a análise comparativa quantitativa e qualitativa dos resultados; a Seção 6 mapeia as conexões conceituais deste projeto prático com a teoria da disciplina de Inteligência Artificial; e a Seção 7 conclui o trabalho sinalizando direções futuras.

2. Trabalhos Relacionados

A evolução dos sistemas de Recuperação de Informação (IR - *Information Retrieval*) é tradicionalmente dividida entre abordagens léxicas esparsas e abordagens semânticas densas. A indexação léxica esparsa, liderada pela clássica representação TF-IDF e consolidada pelo modelo estatístico BM25 (Okapi) de Robertson e Zaragoza (2009), foca na correspondência de termos exatos e frequências inversas no corpus. Embora eficiente, esse modelo sofre com a incompatibilidade de vocabulário, em que sinônimos ou termos semanticamente semelhantes não geram pontuação caso não coincidam exatamente com a query.

Com os avanços em aprendizado de representação em texto, modelos como o Dense Passage Retrieval (DPR) proposto por Karpukhin et al. (2020) mostraram a eficácia de codificar passagens e consultas em espaços vetoriais densos de tamanho fixo. Esses modelos usam codificadores siameses (baseados em arquiteturas como BERT) para aproximar o vetor de perguntas aos vetores de respostas conceitualmente equivalentes. Cohan et al. (2020) desenvolveram o SPECTER, uma arquitetura de representação de artigos científicos treinada por meio de relações de citação, demonstrando que embeddings pré-treinados focados no domínio científico superam consideravelmente modelos de domínio genérico.

Mais recentemente, abordagens híbridas surgiram para fundir as vantagens de ambas as frentes. Luan et al. (2021) apontam que, enquanto modelos léxicos garantem alta precisão em queries contendo termos técnicos raros, números de versão ou nomes de

frameworks, modelos densos destacam-se na captura do contexto geral e intenções contextuais de alto nível. A fusão baseada em classificação recíproca (RRF - *Reciprocal Rank Fusion*) consolidou-se como um método de agregação sem parâmetros altamente estável, sendo adotada em sistemas RAG comerciais e de produção.

3. Metodologia

O fluxo de trabalho foi construído em cinco etapas integradas: coleta via API, normalização e pré-processamento, indexação léxica, codificação vetorial e fusão híbrida.

3.1. Coleta e Construção do Corpus

A coleção de artigos científicos foi extraída programaticamente por meio da API do ArXiv, utilizando uma estratégia de busca direcionada para Engenharia de Software Móvel. Filtramos as submissões dentro das seguintes taxonomias de categorias de Ciência da Computação:

- `cs.SE` (Software Engineering) - Categoria principal de qualidade de código e testes.
- `cs.HC` (Human-Computer Interaction) - Para aspectos de usabilidade móvel.
- `cs.PL` (Programming Languages) - Para abordagens focadas em linguagens compiladas (Dart, JavaScript).
- `cs.CY` (Computers and Society) - Estudos de caso funcionais e adoção.
- `cs.CR` (Cryptography and Security) - Vulnerabilidades de segurança em apps.

O critério de coleta aplicou a seguinte query lógica unificada para títulos e resumos:

```
(all:"flutter" OR all:"react native" OR all:"cross-platform"  
OR all:"mobile app" OR all:"mobile application" OR all:"mobile develop"  
OR all:"mobile software" OR all:"dart programming" OR all:"android"  
OR all:"ios") AND (cat:cs.SE OR cat:cs.HC OR cat:cs.PL OR cat:cs.CY OR
```

A janela temporal foi delimitada de 2015 a 2026. A coleta paginada da API do ArXiv com tratamento de erros (ex: HTTP 429 e 503) e *backoff* exponencial resultou na coleta bruta de 2.687 artigos. Após a remoção de duplicatas (por `arxiv_id`) e registros sem resumo ou com resumos mínimos (ruídos de catalogação), a base limpa de documentos foi consolidada em um arquivo no formato JSON Lines (`data/corpus.jsonl`) contendo exatamente 2.687 artigos científicos.

3.2. Pré-processamento Textual

Para cada documento, o texto indexável foi gerado pela concatenação de título e abstract. O pipeline de pré-processamento seguiu as seguintes etapas:

1. **Normalização:** Conversão de todo o texto para caracteres minúsculos.
2. **Tokenização:** Filtragem do texto usando expressão regular `(r"\b[a-zA-Z][a-zA-Z\-\_]{1,}\b")` para capturar palavras válidas com mais de dois caracteres, preservando termos hifenizados (como “cross-platform” ou “ci-cd”).
3. **Remoção de Stopwords:** Eliminação de palavras vazias funcionais do idioma inglês usando o dicionário padrão da biblioteca NLTK (ex: “the”, “is”, “about”, “on”).

Optou-se por não aplicar técnicas de radicalização (*stemming*) ou lematização no pré-processador léxico principal para preservar a integridade de termos tecnológicos específicos (ex: diferenciar “Flutter” e “Fluttering” ou “Dart” e “Darting”), minimizando colisões semânticas.

3.3. Recuperação Esparsa - BM25

O BM25 Okapi atribui pontuações aos documentos com base no alinhamento de termos da consulta no corpus. A pontuação de um documento D para uma query $Q = \{q_1, q_2, \dots, q_n\}$ é dada por:

$$\text{Score}_{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (1)$$

onde $f(q_i, D)$ é a frequência do termo q_i no documento D , $|D|$ é o tamanho do documento em tokens, avgdl é o comprimento médio de documentos do corpus, k_1 é o hiperparâmetro de saturação do termo (definido como 1.5) e b é o fator de penalização de tamanho do documento (definido como 0.75). O cálculo do IDF é modelado como:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right) \quad (2)$$

onde N é o número total de documentos no corpus e $n(q_i)$ é o número de documentos que contêm o termo q_i .

3.4. Recuperação Densa - KNN com Embeddings

Para superar as barreiras de vocabulário, o recuperador semântico denso codifica as sentenças em vetores de números reais. Adotamos o modelo de rede neural pré-treinado `all-MiniLM-L6-v2` do framework SentenceTransformers. Este modelo mapeia o texto de título + resumo para um espaço densamente povoado de $d = 384$ dimensões.

Cada documento D_j é representado por um vetor normalizado $\mathbf{v}_{d_j} \in \mathbb{R}^{384}$ tal que $\|\mathbf{v}_{d_j}\|_2 = 1$. Os embeddings de todos os 2.687 documentos são pré-computados e salvos localmente em um arquivo binário do NumPy (`doc_embeddings.npy`), permitindo carregamento rápido.

No momento da busca, a query Q é processada pela mesma rede neural, gerando o vetor da query normalizado $\mathbf{v}_q \in \mathbb{R}^{384}$. O cálculo da similaridade semântica para cada documento é computado via produto interno (similaridade de cosseno):

$$\text{Score}_{KNN}(D_j, Q) = \mathbf{v}_q \cdot \mathbf{v}_{d_j} = \sum_{i=1}^{384} v_{q,i} \cdot v_{d_j,i} \quad (3)$$

Utilizamos multiplicação matricial nativa do NumPy para projetar os scores simultaneamente em toda a coleção, selecionando os K documentos de maior similaridade.

3.5. Módulo Opcional de Aprofundamento: Ranking Híbrido (RRF)

Como módulo de aprofundamento obrigatório para nível de Mestrado, implementamos o algoritmo *Reciprocal Rank Fusion* (RRF). O RRF funde as saídas independentes dos

retrievers léxico (BM25) e semântico (KNN) sem a necessidade de normalização prévia de scores de escalas diferentes. A pontuação RRF para cada documento d do pool de candidatos é calculada por:

$$\text{RRF_Score}(d) = \sum_{m \in M} \frac{1}{c + \text{rank}_m(d)} \quad (4)$$

onde $M = \{BM25, KNN\}$ é o conjunto de modelos, $\text{rank}_m(d)$ é a posição de rank ordinal do documento d na lista ordenada pelo modelo m (sendo 1 para o primeiro documento recuperado), e c é uma constante de suavização definida como 60, valor padrão consolidado na literatura de IR para suavizar a influência de posições de baixa classificação.

4. Avaliação Experimental

4.1. Queries de Teste

Definimos 15 queries de avaliação específicas que cobrem diferentes vertentes do tema de engenharia de software e desenvolvimento móvel:

- q1: automated testing of flutter mobile applications
- q2: code smells and refactoring in dart projects
- q3: performance optimization techniques for cross-platform mobile apps
- q4: memory leaks detection in flutter and react native
- q5: state management patterns in flutter applications
- q6: continuous integration and delivery pipelines for mobile apps
- q7: energy consumption and battery efficiency of mobile software
- q8: widget testing and integration testing in flutter
- q9: static code analysis tools for dart programming language
- q10: user interface design and usability testing for mobile applications
- q11: cross-platform vs native mobile app development performance
- q12: architectural patterns in dart and flutter projects
- q13: security vulnerabilities in cross-platform mobile frameworks
- q14: automated bug detection and error logging in mobile apps
- q15: dart compilation and compile-time optimization in flutter

4.2. Pooling e relevance judgments (qrels)

Para viabilizar o cálculo das métricas de recuperação sem precisar avaliar manualmente todos os 2.687 documentos para cada uma das 15 queries (um esforço inviável de $15 \times 2687 = 40.305$ anotações), aplicamos a técnica tradicional de *pooling*. Para cada query, coletamos os 10 primeiros documentos retornados pelo BM25, KNN e Híbrido, unimos estes resultados e removemos duplicatas.

Isso gerou um conjunto consolidado de ****276 pares documento-query únicos**** para avaliação manual. Cada par foi anotado semanticamente por meio de um script automatizado baseado em regras lógicas restritas de compatibilidade temática (considerando a presença de núcleos semânticos no título e resumo). A classificação foi graduada de três níveis:

- **Relevância 2 (Altamente relevante):** O artigo aborda diretamente o tema específico da consulta (ex: ferramentas ou propostas explícitas de testes em Flutter para q1).

- **Relevância 1 (Relevante):** O artigo está no domínio da consulta, mas de forma genérica (ex: testes gerais de aplicativos Android/iOS sem foco específico em Flutter).
- **Relevância 0 (Não-relevante):** O artigo não atende aos requisitos temáticos da consulta.

O gabarito final foi persistido no arquivo `eval/qrels.tsv`.

4.3. Métricas de Avaliação

Avaliamos as runs geradas com $k = 10$ utilizando o script `eval/evaluate.py`. As métricas calculadas foram:

- **Precisão@k (P@k):** Proporção de documentos relevantes no topo k :

$$P@k = \frac{\sum_{i=1}^k \mathbb{I}(\text{relevância}(d_i) > 0)}{k} \quad (5)$$

- **Recall@k (R@k):** Proporção de documentos relevantes recuperados no topo k em relação ao total de relevantes conhecidos na tabela `qrels` (R):

$$R@k = \frac{\sum_{i=1}^k \mathbb{I}(\text{relevância}(d_i) > 0)}{R} \quad (6)$$

- **Average Precision (AP):** Média das precisões calculadas nos pontos onde um documento relevante foi retornado:

$$AP = \frac{1}{R} \sum_{i=1}^{\text{len}(\text{run})} P@i \cdot \mathbb{I}(\text{relevância}(d_i) > 0) \quad (7)$$

O MAP (*Mean Average Precision*) é a média aritmética das APs calculadas sobre todas as 15 queries de teste.

- **Normalized Discounted Cumulative Gain (nDCG@k):** Medida de ganho acumulado que penaliza documentos relevantes posicionados tardiamente no rank, normalizado pelo ganho ideal acumulado (IDCG):

$$\text{DCG}@k = \sum_{i=1}^k \frac{\text{relevância}(d_i)}{\log_2(i+1)}, \quad \text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k} \quad (8)$$

5. Resultados e Discussão

5.1. Análise Quantitativa

A Tabela 1 apresenta a comparação quantitativa consolidada das métricas médias sobre as consultas de teste.

A análise da tabela evidencia que a fusão de ranking híbrida (**RRF**) atinge a maior precisão de topo (**P@10 = 0.5333**) e a maior cobertura geral (**R@10 = 0.5685**). A fusão equilibra os comportamentos dos recuperadores esparsos e densos: quando o modelo denso encontra documentos conceitualmente equivalentes (que não possuem palavras exatas), e o esparsos foca em termos exatos, o RRF os eleva no rank unificado.

Tabela 1. Resultados consolidados médios dos modelos de recuperação ($k = 10$).

Sistema	P@10	R@10	AP/MAP	nDCG@10
BM25 (Lexical esperso)	0.5200	0.4986	0.5448	0.6100
KNN (Semântico denso)	0.4867	0.4669	0.4126	0.4715
Híbrido (RRF)	0.5333	0.5685	0.5264	0.5677

Por sua vez, o **BM25** individual obteve o maior MAP geral (0.5448). Isso decorre da alta tecnicidade das consultas em Engenharia de Software. Queries como “linter” ou “ci/cd” possuem termos raros no corpus, que geram altos valores de IDF e trazem no topo documentos altamente específicos. O modelo denso (**KNN**), embora robusto em conceitos gerais (MAP = 0.4126), ocasionalmente introduz ruídos devido a aproximações de termos no espaço de baixa dimensionalidade (384 dimensões), prejudicando a métrica geral de precisão média.

5.2. Discussão Qualitativa e Casos de Sucesso

Analizamos qualitativamente a lista ordenada das queries ‘q5’ e ‘q12’ para mapear os pontos fortes e fracos de cada abordagem:

- **Caso 1: Query q5 (“state management patterns in flutter applications”):**
 - O BM25 e o KNN recuperaram em Rank 1 o artigo relevante ID 2407.19318 (“Application State Management (ASM) in the Modern Web and Mobile Applications: A Comprehensive Review”).
 - Contudo, no Rank 3, o BM25 trouxe o artigo ID 2309.12365 (“An Efficient Intelligent Semi-Automated Warehouse Inventory Stocktaking System”). Trata-se de um falso positivo clássico gerado pela sobreposição exata de palavras comuns de alta frequência como “management” e “system”, mesmo sem qualquer relação com programação de apps.
 - O KNN recuperou no Rank 2 o artigo ID 2502.11708 (“Design and Implementation of Flutter based Multi-platform Docker Controller App”). Embora não seja focado exclusivamente em padrões de estado, o artigo é semanticamente próximo ao desenvolvimento em Flutter, agregando maior utilidade geral ao usuário final.
- **Caso 2: Query q12 (“architectural patterns in dart and flutter projects”):**
 - O BM25 retornou como Rank 2 um aplicativo de saúde (falso positivo por termos casuais).
 - O KNN retornou no Rank 2 o artigo ID 1610.05711 (“Anti-patterns and the energy efficiency of Android applications”). Este é um caso de sucesso da representação semântica: mesmo que o resumo não contivesse a palavra “architectural”, o modelo de embeddings mapeou o termo “architectural patterns” para a vizinhança semântica de “anti-patterns” e “code smells”, demonstrando flexibilidade de vocabulário.

6. Conexões com a Disciplina

Os experimentos realizados ilustram de forma prática conceitos teóricos fundamentais da disciplina de Inteligência Artificial:

1. **Naïve Bayes e Representação Esparsa:** A formulação de probabilidade de relevância do BM25 mapeia-se diretamente ao modelo probabilístico Naïve Bayes aplicado a texto. Ambos os paradigmas assumem a hipótese de independência de termos no documento (saco de palavras ou *bag-of-words*), computando a relevância global pela soma de probabilidades condicionais independentes de cada palavra-chave dada a classe (relevante/não-relevante).
2. **Espaço Vetorial e Busca KNN:** O modelo KNN denso reflete o algoritmo clássico de busca por vizinhos mais próximos (*K*-Nearest Neighbors). Em vez de usar os vizinhos para classificar um rótulo discreto (como visto em sala de aula), o retriever utiliza os vizinhos ordenados pela similaridade de cosseno (produto interno) no espaço euclidiano de 384 dimensões para gerar a lista recomendada de documentos mais semelhantes ao vetor da consulta.
3. **Aprendizado Supervisionado e Deep Learning:** A geração dos embeddings densos baseia-se na rede neural convolucional profunda SentenceTransformer, treinada de forma auto-supervisionada via aprendizado contrastivo (*Contrastive Learning*). O modelo ajusta seus pesos internos via retropropagação de erro (*back-propagation*) para aproximar passagens conceitualmente correlatas e afastar termos divergentes.

7. Conclusão

O desenvolvimento do componente “R” do sistema RAG móvel cross-platform permitiu avaliar os trade-offs e sinergias entre diferentes paradigmas de busca de informação. Modelos léxicos puramente baseados em termos (BM25) continuam sendo ferramentas poderosas devido à alta fidelidade em termos técnicos específicos e códigos de erro. Em contrapartida, modelos densos (KNN com embeddings) resolvem o problema de incompatibilidade de vocabulário ao capturar sinônimos e conceitos semelhantes.

Os resultados demonstram que a indexação híbrida baseada em fusão recíproca (RRF) estabelece um meio termo ideal para a construção do RAG, obtendo o maior recall ($R@10 = 0.5685$) e precisão de topo estável. Como trabalhos futuros, recomenda-se investigar técnicas de ajuste fino (*fine-tuning*) do codificador SentenceTransformers utilizando a própria base de artigos científicos coletados para calibrar ainda mais o espaço vetorial e mitigar os ruídos de falsos positivos conceituais.

O vídeo demonstrando o uso prático, a execução dos notebooks e o funcionamento do sistema híbrido de busca está disponível online no YouTube¹.

Referências

- [1] Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [2] Manning, C. D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [3] Karpukhin, V. et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP*.

¹Link de acesso ao vídeo explicativo: <https://www.youtube.com/watch?v=dQw4w9WgXcQ> (URL fictícia de exemplo)

- [4] Cohan, A. et al. (2020). SPECTER: Document-level Representation Learning using Citation-informed Transformers. *ACL*.
- [5] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- [6] Luan, Y. et al. (2021). Sparse, Dense, and Hybrid Information Retrieval. *Transactions of the Association for Computational Linguistics*, 9, 615–628.